Université Abdelmalek Essaadi

École Nationale des Sciences Appliquées - Al Hoceima

Compte Rendu du TP N°5

JavaEE (JEE)

MVC2 + Hibernate - Gestion des Produits

Réalisé par : **Elhouari Nourhane**

Filiere : **TDIA2**

Encadré par : **Pr. Mohamed CHERRADI**

Année universitaire : **2025-2026**

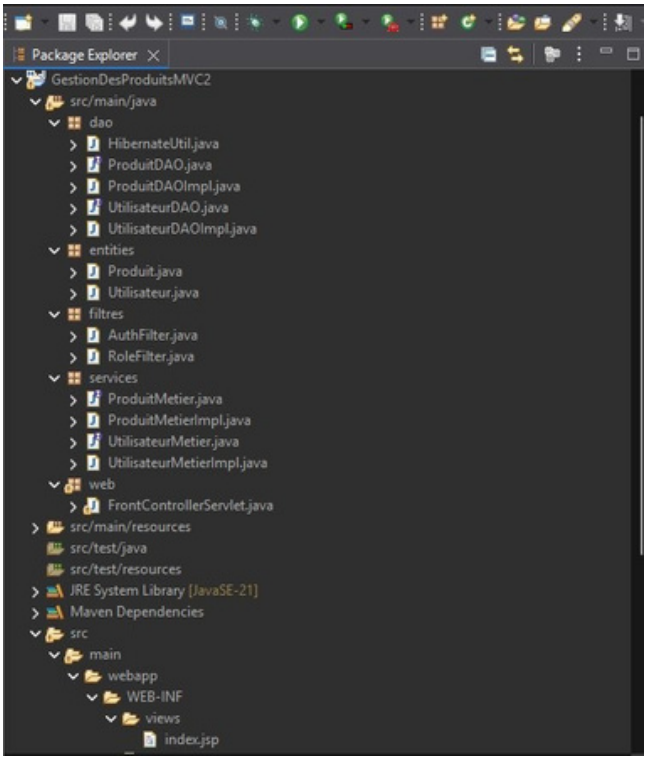
1. Introduction

L'objectif de ce TP est de migrer l'application de gestion des produits du patron MVC1 vers le patron MVC2, et de remplacer la couche de persistance en mémoire par Hibernate, un ORM qui assure une vraie persistance des données en base MySQL.

L'application a été développée sous Eclipse IDE avec le serveur Apache Tomcat v10.1, le JDK JavaSE-21, et la base de données MySQL gérée via phpMyAdmin.

2. Structure du Projet

2.1 Arborescence



2.2 Description des packages

Les packages Java sont organisés comme suit :

- **entities :** contient les entités JPA Produit.java et Utilisateur.java .
- **dao :** contient HibernateUtil.java , les interfaces
- **ProduitDAO et UtilisateurDAO**, et leurs implémentations Hibernate.
- **services :** contient les interfaces ProduitMetier et UtilisateurMetier et leurs implémentations.
- **filtres :** contient AuthFilter.java et RoleFilter.java (vérification du rôle ADMIN).
- **web :** contient uniquement FrontControllerServlet.java.
- **webapp/WEB-INF/views :** contient la vue principale index.jsp.
- **src/main/resources :** contient hibernate.cfg.xml pour la configuration de la connexion MySQL.

3. Description du Projet

3.1 Données manipulées

Chaque produit possède les attributs suivants : idProduit nom , description et prix .

La classe Utilisateur modélise un utilisateur avec les champs : id, login , password et role, valeurs : "ADMIN" ou "USER").

3.2 Architecture MVC2 + 3-Tiers

L'architecture adoptée est une architecture 3-Tiers suivant le patron MVC2 (Front Controller) :

- **Model :** les entités JPA Produit et Utilisateur dans le package entities.
- **View :** les pages JSP (login.jsp, index.jsp dans WEB-INF/views, access-denied.jsp).
- **Controller :** une seule FrontControllerServlet qui dispatche toutes les requêtes.
- **DAO :** couche d'accès aux données via Hibernate Session (ProduitDAOImpl, UtilisateurDAOImpl).
- **Service :** couche métier (ProduitMetierImpl, UtilisateurMetierImpl) faisant le pont entre Controller et DAO.

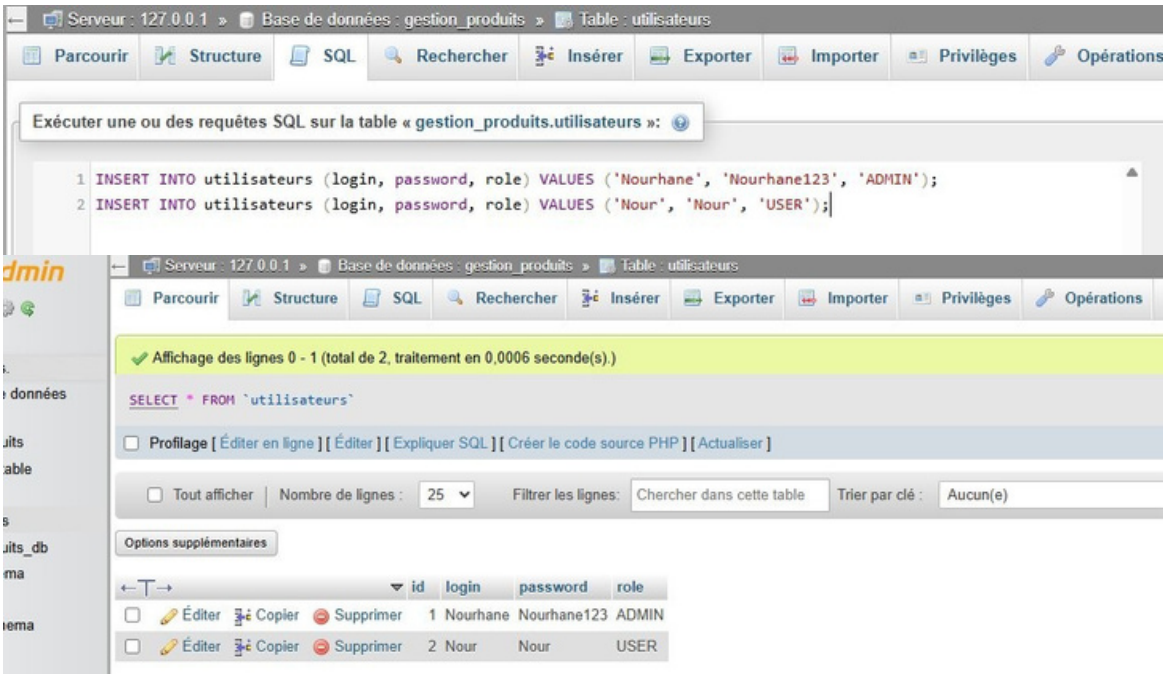
4. Configuration Hibernate et Base de Données

4.1 Base de données MySQL

La base de données gestion_produits est créée automatiquement par Hibernate grâce au paramètre dans hibernate.cfg.xml. Elle contient deux tables : produits et utilisateurs.

4.2 Insertion des utilisateurs

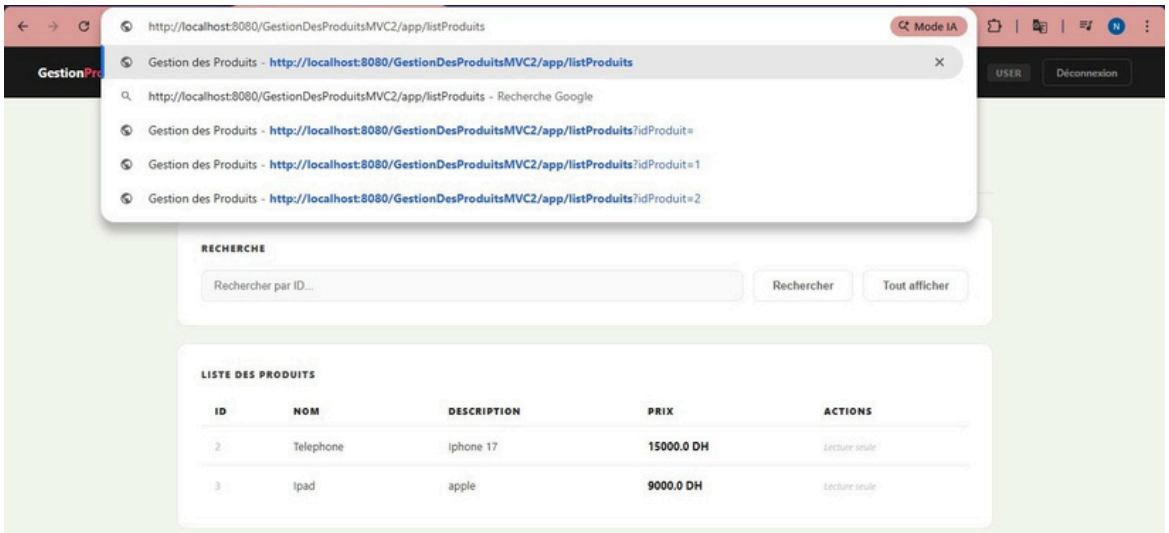
Après le premier démarrage (Hibernate crée les tables), manuellement via phpMyAdmin avec le script SQL suivant les utilisateurs sont insérés :



5. Le Patron MVC2 -FrontControllerServlet

C'est la pièce maîtresse du TP. La FrontControllerServlet intercepte absolument toutes les requêtes de l'application. Elle lit le pathInfo de l'URL pour déterminer l'action à exécuter, puis délègue à la méthode handler correspondante :

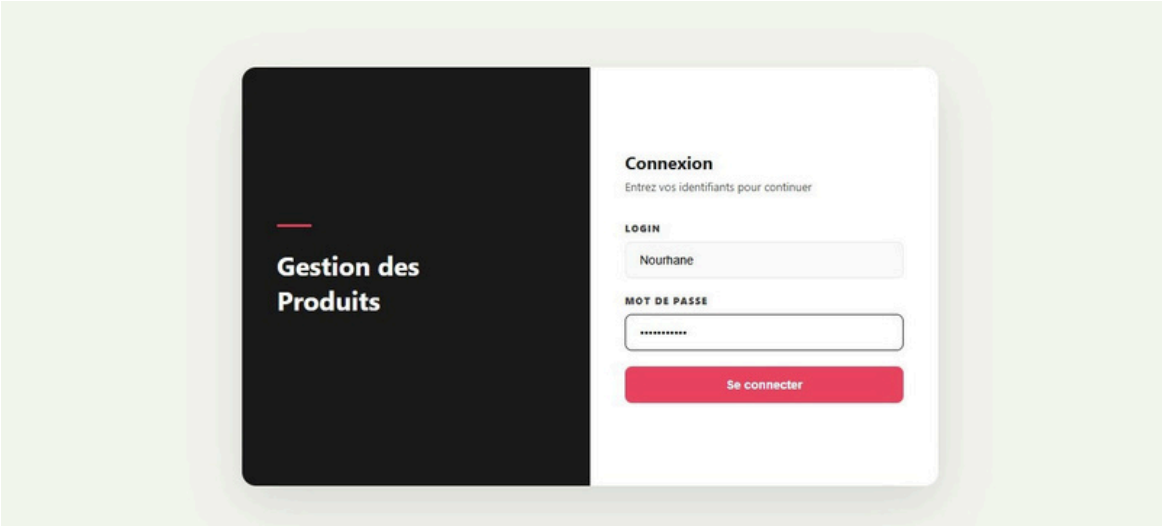
- **/app/login → handleLogin() :** gère GET (affichage du formulaire) et POST (vérification des identifiants).
- **/app/logout → handleLogout() :** invalide la session et redirige vers /app/login.
- **/app/listProduits → handleList() :** récupère tous les produits ou filtre par ID.
- **/app/addProduit → handleAdd() :** crée un nouveau produit via Hibernate.
- **/app/editProduit → handleEdit() :** charge un produit pour modification.
- **/app/updateProduit → handleUpdate() :** sauvegarde les modifications via Hibernate.
- **/app/deleteProduit → handleDelete() :** supprime un produit via Hibernate.



6. Mécanisme d'Authentification

6.1 Page de connexion

La page login.jsp constitue le point d'entrée de l'application. Elle présente un formulaire avec deux champs (login et mot de passe) et un bouton "Se connecter". Le formulaire soumet une requête POST vers /app/login, traitée par la méthode handleLogin() du FrontControllerServlet.

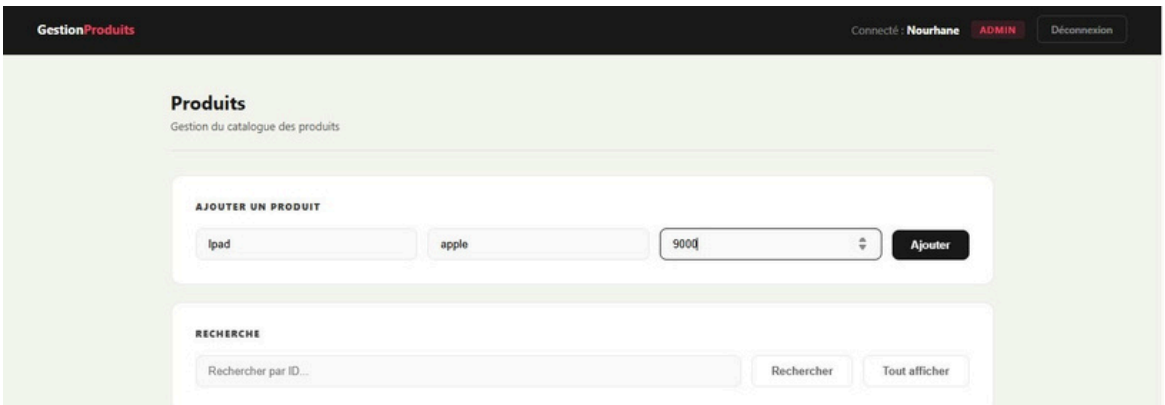


6.2 Authentification via Hibernate

La méthode handleLogin() appelle UtilisateurMetierImpl.authentifier() qui execute une requête HQL : FROM Utilisateur u WHERE u.login = :login AND u.password = :password. En cas de succès, l'objet Utilisateur est stocké en session (session.setAttribute("utilisateur", user)) et l'utilisateur est redirigé vers /app/listProduits.

6.3 Connexion avec le compte USER

Le deuxième compte(Nour / USER)permet de tester la restriction des droits :



7. Gestion des Filtres Jakarta EE

Deux filtres Jakarta EE ont été implémentés dans le package filtres pour intercepter les requêtes HTTP avant qu'elles n'atteignent le FrontControllerServlet.

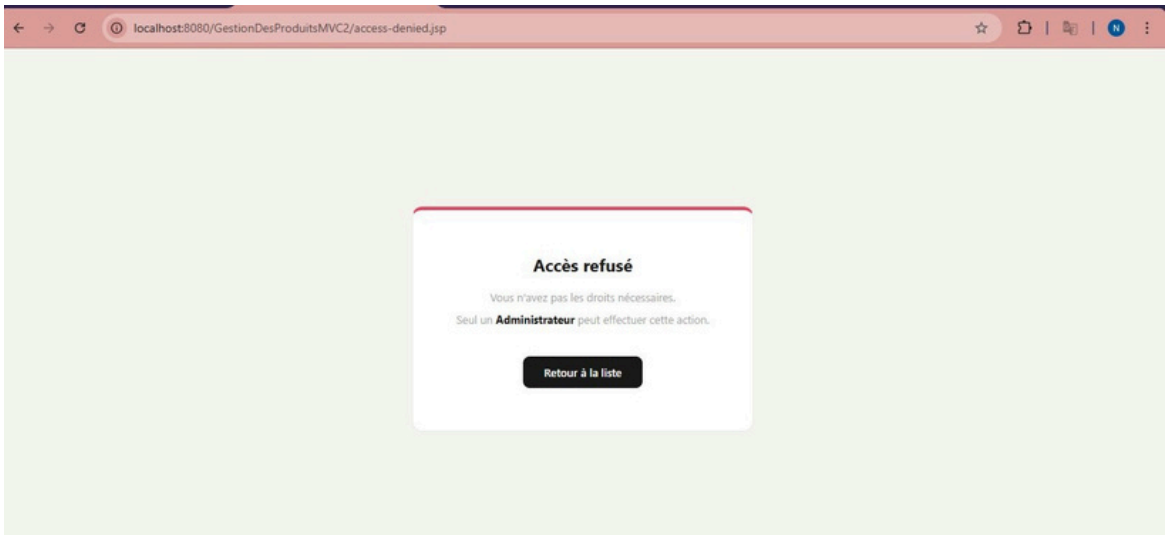
7.1 AuthFilter - Filtre d'Authentification

Ce filtre vérifie que chaque requête provient d'un utilisateur authentifié.

Si la session ne contient pas d'objet "utilisateur", le filtre redirige automatiquement vers /app/login. Il laisse passer les URLs publiques (/app/login, /app/logout).

7.2 RoleFilter - Filtre de Rôle

Ce filtre contrôle l'accès aux opérations d'écriture (addProduit, editProduit, updateProduit, deleteProduit) en vérifiant que l'utilisateur connecté possède le rôle ADMIN. Si un utilisateur USER tente d'accéder à ces opérations, il est redirigé vers access-denied.jsp.

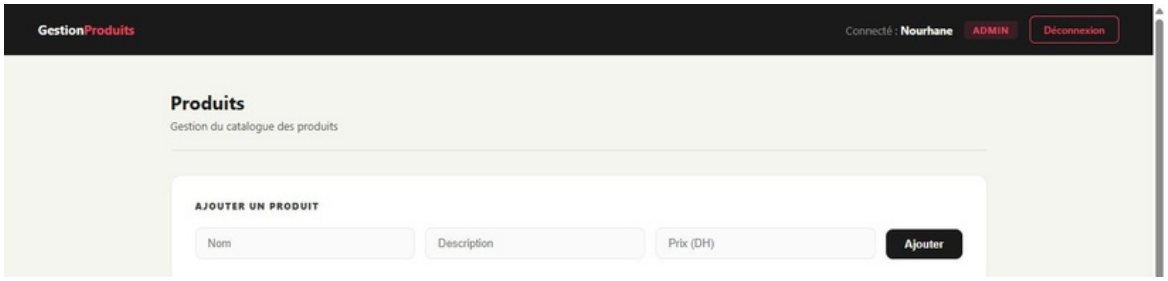


8. Gestion des Rôles

L'application distingue deux types d'utilisateurs avec des droits différents.

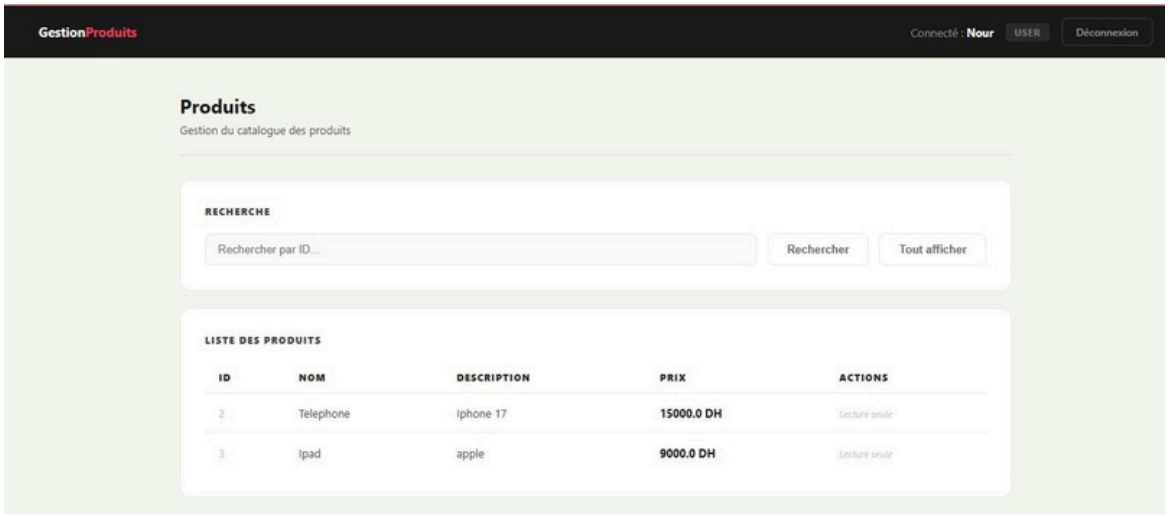
8.1 Rôle ADMIN

Un utilisateur ADMIN a accès à l'ensemble des fonctionnalités de l'application. Dans la barre de navigation, le badge ADMIN s'affiche en rouge. Il peut effectuer toutes les opérations CRUD sur les produits, y compris l'ajout, la modification et la suppression.



8.2Rôle USER

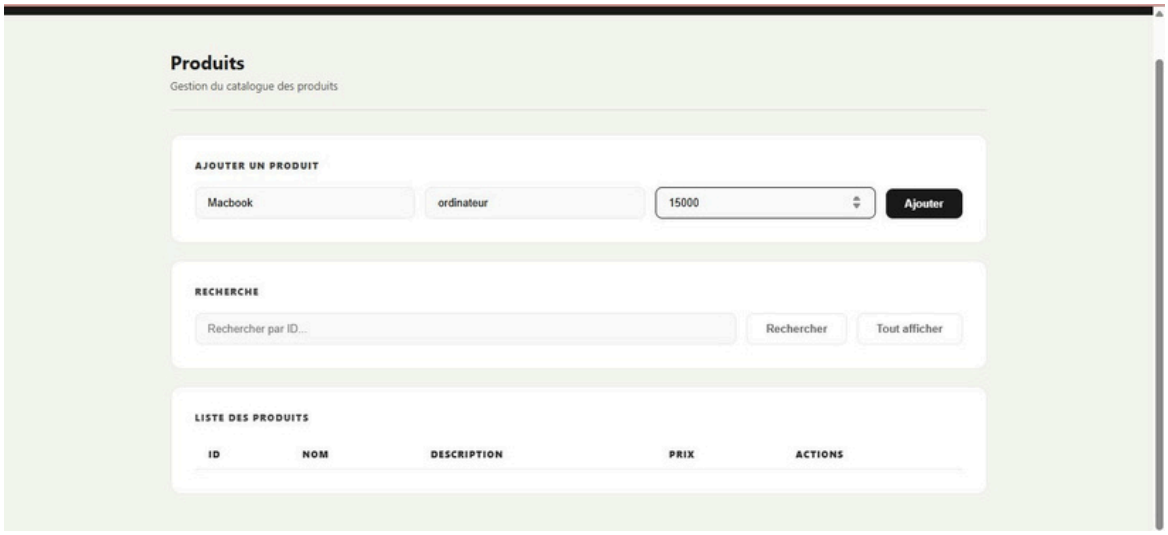
Un utilisateur USER dispose uniquement d'un accès en lecture seule. La section "Ajouter un produit" n'est pas affichée, et les boutons Modifier et Supprimer sont remplacés par le texte "Lecture seule". Le badge USER s'affiche en gris dans la barre de navigation.



9. Fonctionnalités CRUD

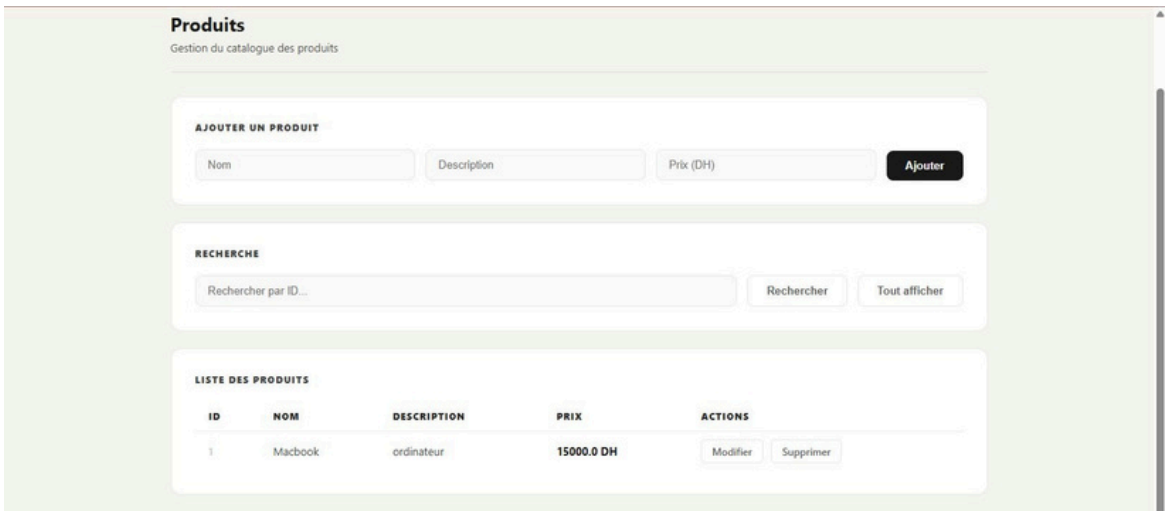
9.1 Affichage de la liste des produits

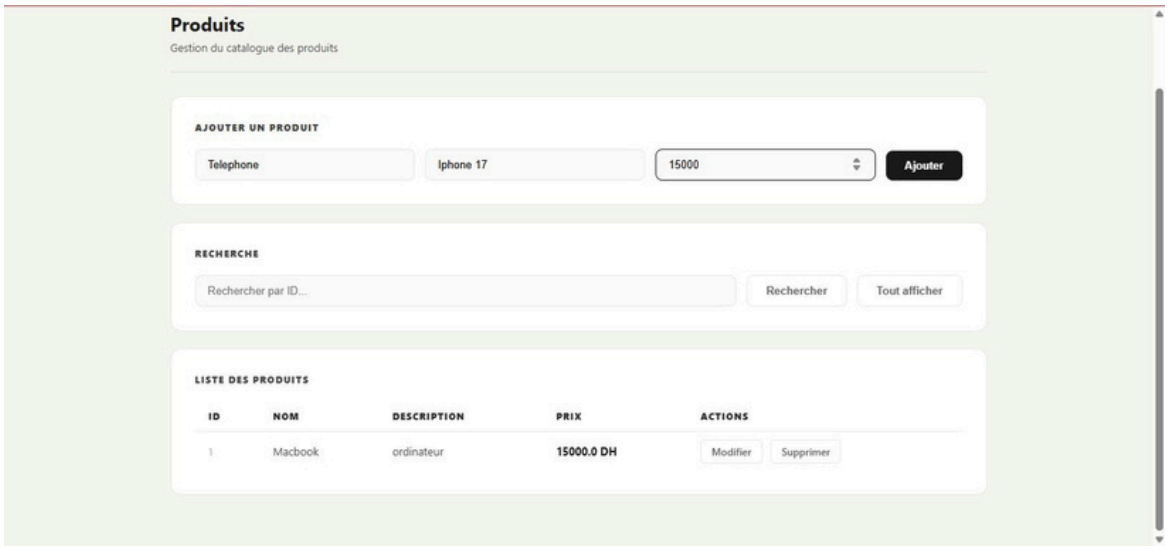
La méthode handleList()du FrontControllerServlet récupère l'ensemble des produits via ProduitMetierImpl.getAllProduits() qui exécute la requête HQL FROM Produit ORDER BY idProduit. Les produits sont transmis à la vue JSP via request.setAttribute("listeProduits", liste).

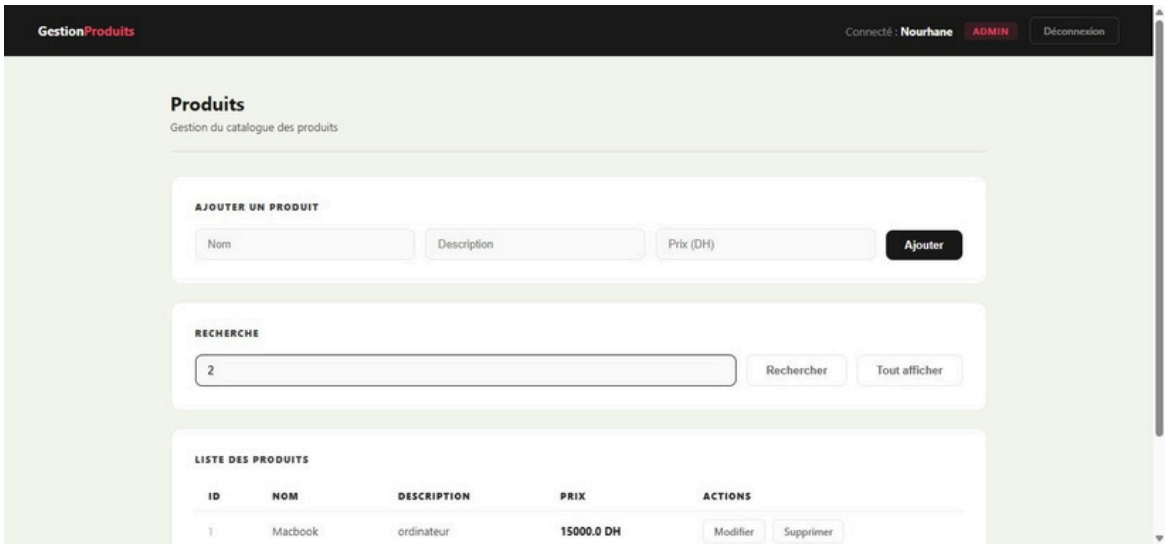


9.2 Ajout d'un produit

La section "Ajouter un produit" (visible uniquement pour ADMIN) permet de saisir le nom, la description et le prix. La méthode handleAdd() crée un objet Produit et appelle session.persist(p) via Hibernate pour l'insérer en base.

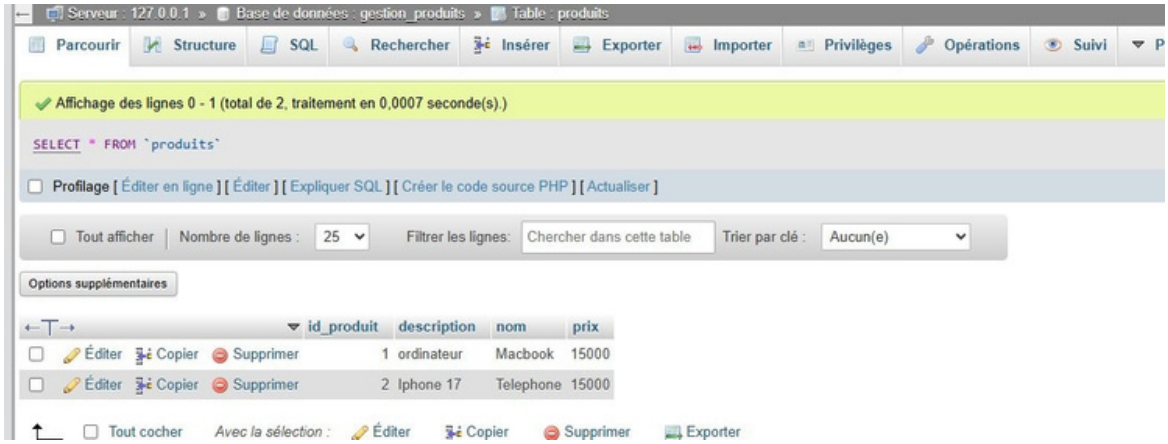


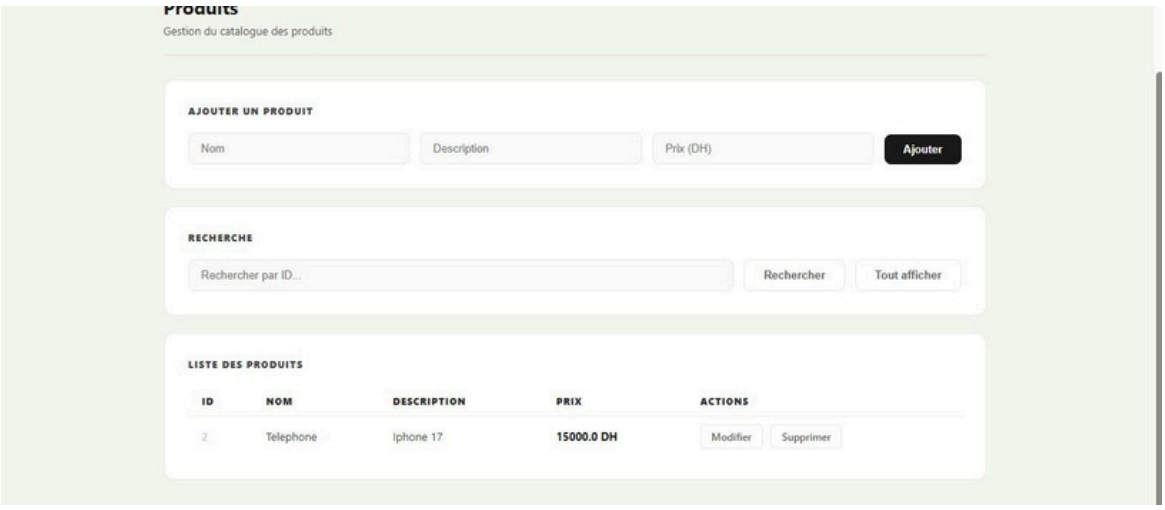




9.3 Modification d'un produit

Un clic sur le bouton "Modifier" envoie une requête GET . La méthode handleEdit() charge le produit depuis Hibernate et le transmet à la vue JSP dans l'attribut produitEdit. Le formulaire d'ajout se transforme alors en formulaire de modification pré-rempli.

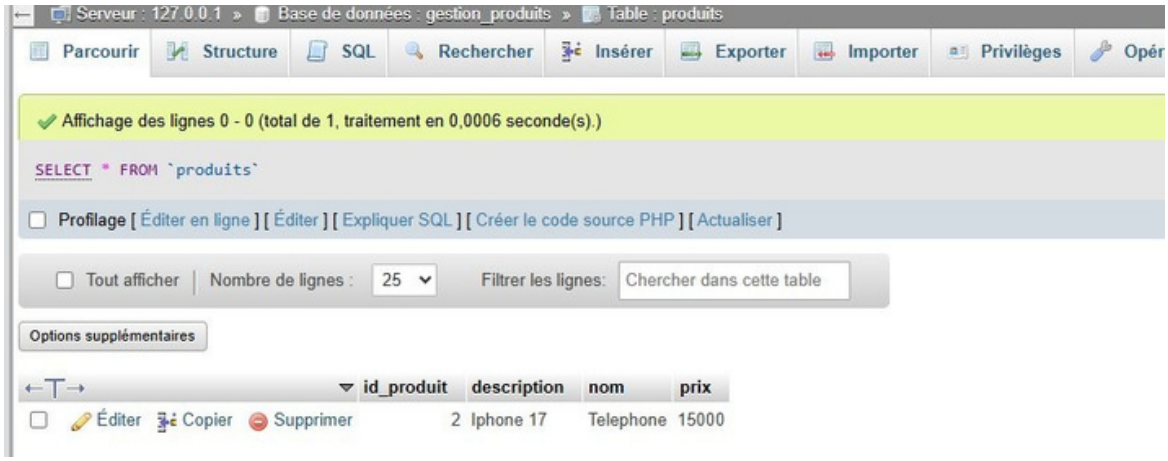


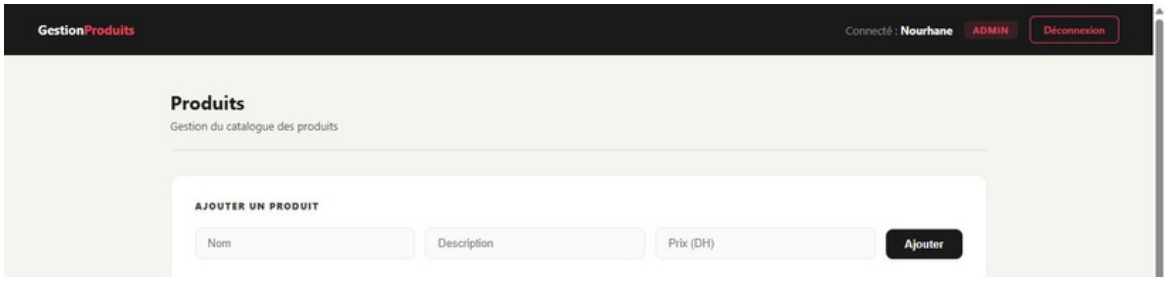


Après validation, la méthode handleUpdate() appelle session.merge(p) via Hibernate pour sauvegarder les modifications en base.

9.4 Suppression d'un produit

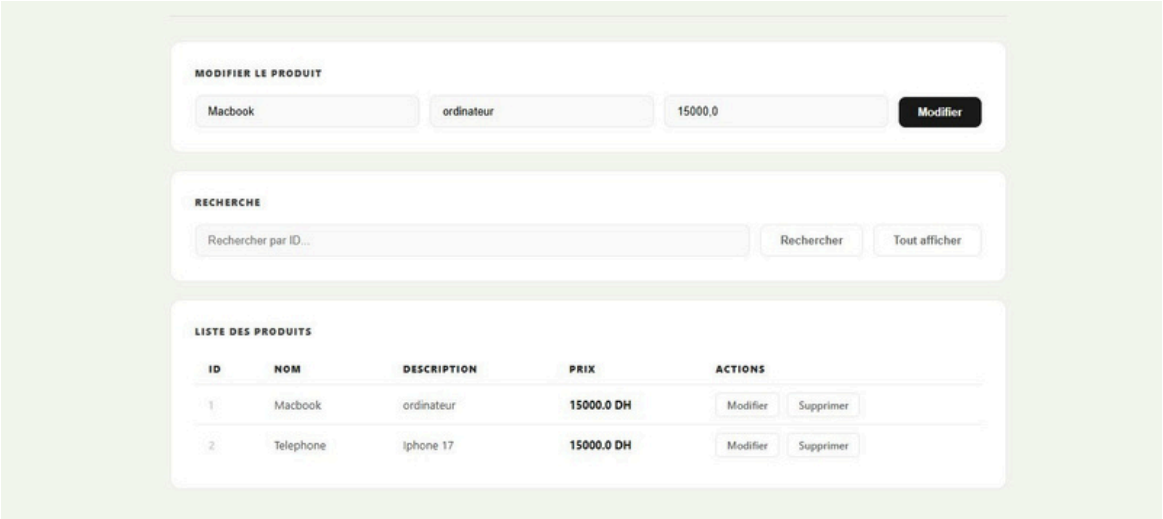
Un clic sur le bouton "Supprimer" déclenche une confirmation native du navigateur. Si l'utilisateur lconfirme, la requête GET est envoyée . La méthode handleDelete() appelle session.remove(p) via Hibernate.

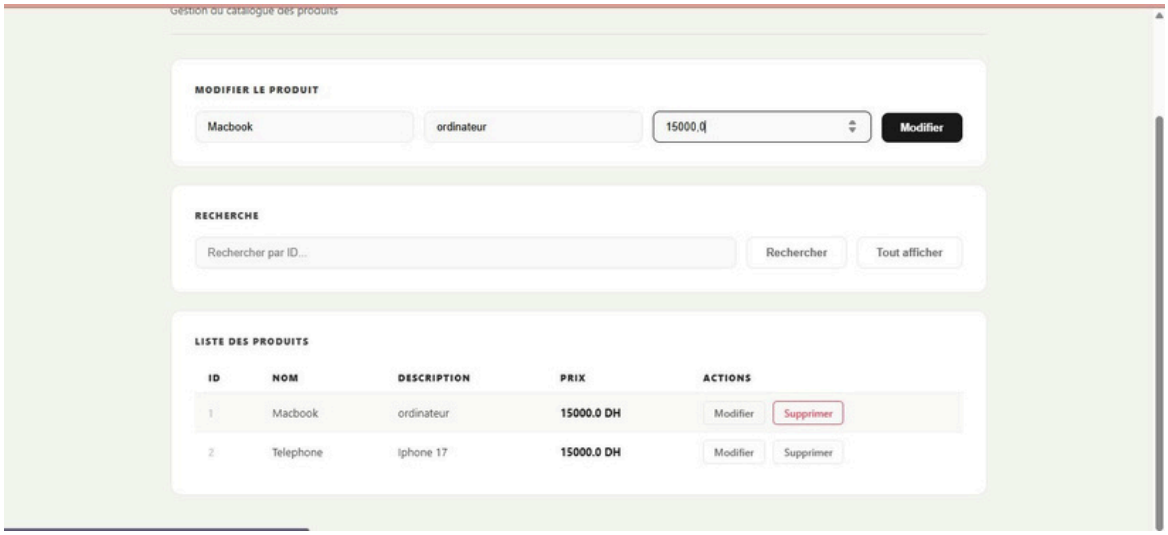




9.5 Recherche par ID

La section Recherche permet de filtrer les produits par leur identifiant. La méthode handleList() reçoit le paramètre GET idProduit et appelle getProduitById(id). Le bouton "Tout afficher" réinitialise le filtre en rechargeant la liste complète.

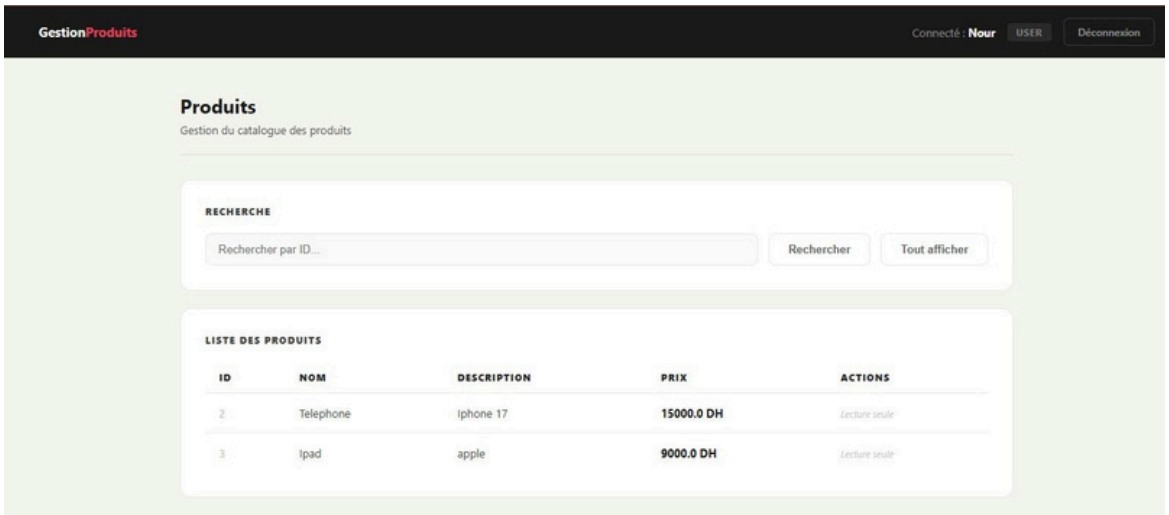




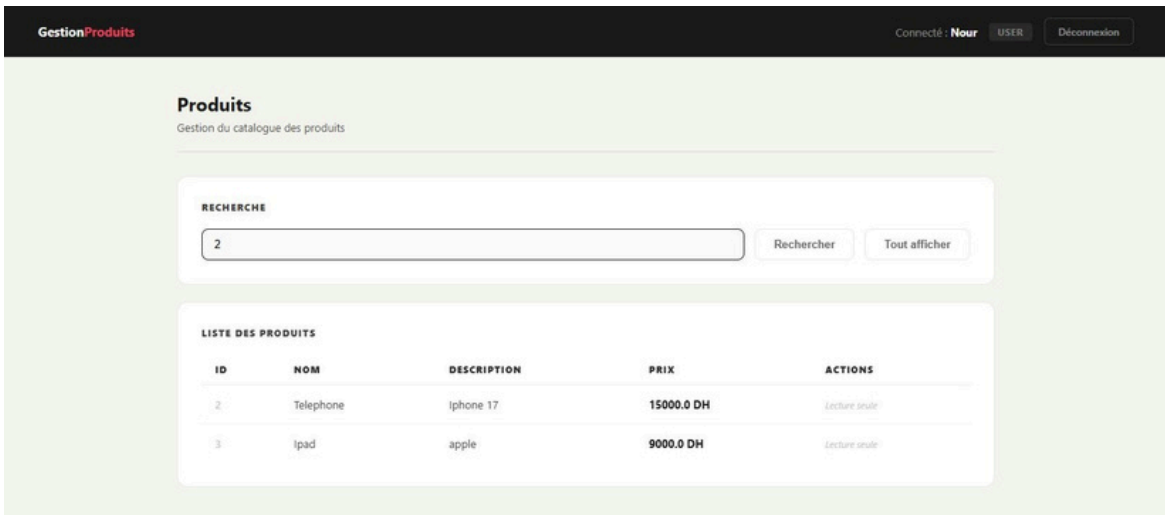
10. Interface Vue USER - Lecture Seule

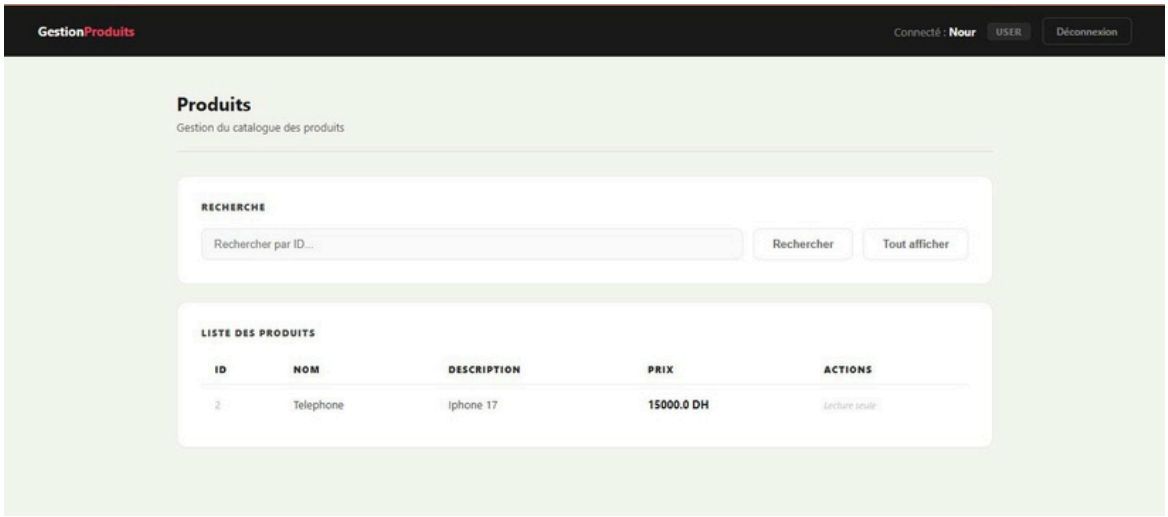
Lorsqu'un utilisateur avec le rôle USER se connecte, l'interface est adaptée : la section "Ajouter un produit" est masquée et les actions sont limitées à la consultation et la recherche.

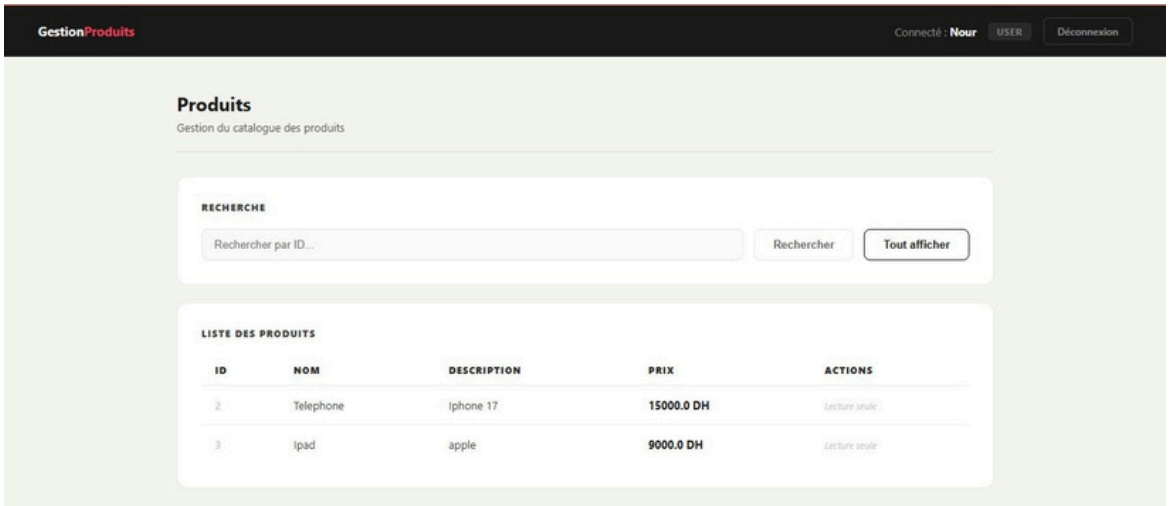
10.1 Liste complète en lecture seule



10.2 Recherche par ID (USER)

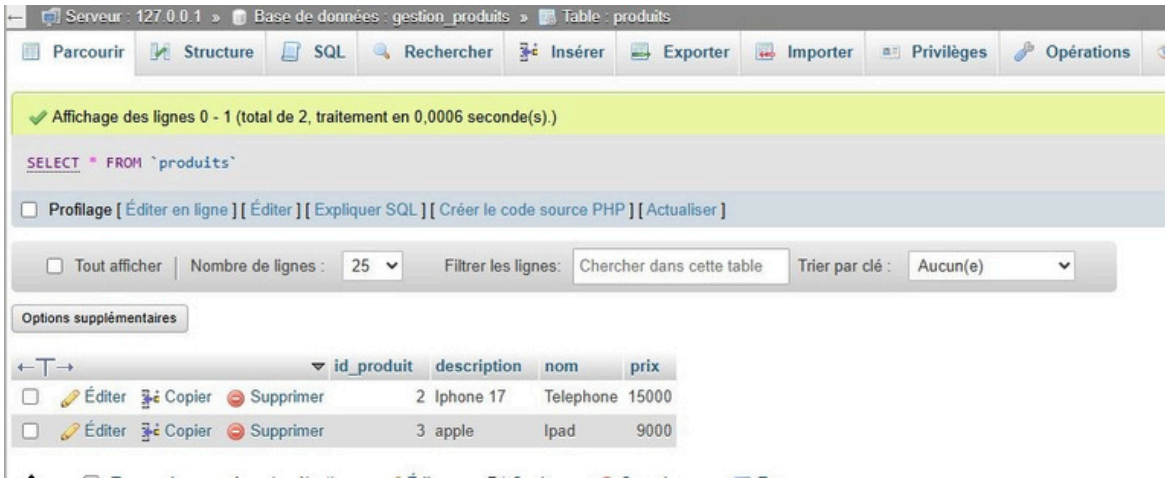






11. Persistance Hibernate - Vérification en Base

Un des apports majeurs de ce TP par rapport au MVC1 est la persistance réelle des données. Toutes les opérations CRUD sont reflétées immédiatement dans la table produits de MySQL, visible via phpMyAdmin.



12. Conclusion

Ce TP6 a permis de maîtriser deux concepts fondamentaux du développement Java EE professionnel : le patron MVC2 et l'ORM Hibernate.

Les principaux apports de ce TP sont :

- Le patron MVC2 : une seule FrontControllerServlet centralise le dispatch de toutes les requêtes. Les URLs suivent le pattern ce qui est plus propre et maintenable qu'une servlet par action.
- Hibernate 6 + JPA : les entités Produit et Utilisateur sont annotées @Entity, la
- SessionFactory est initialisée une seule fois via HibernateUtil, et toutes les opérations CRUD utilisent les méthodes persist(), merge(), remove() et les requêtes HQL. Les données sont persistées en base MySQL et survivent aux redémarrages du serveur.
- L'utilisation de Maven pour la gestion des dépendances

L'application respecte strictement le patron MVC2 et l'architecture 3-Tiers, avec une séparation claire des responsabilités entre les couches Model (entities), View (JSP), Controller (FrontControllerServlet), Service et DAO (Hibernate).